Day 58 – API Management (APIM) – Part 7: Mediation Policies

Class date: 15th September 2026 | Module: SAP API Management – Regular Expression Protection, Mediation Policies

---

One naming correction: policy #7 in your Mediation list was written as **"XML Transformation."** The official SAP API Management policy name is **"XSL Transform"** (it applies an XSLT stylesheet to convert XML into another format – XML, HTML, or plain text). Corrected in Section 3 below. Every other policy name you listed matched SAP's official Mediation Policies documentation exactly.

1. Recap, and one leftover Security policy (start here first)

Up to yesterday: **Security Policies.** Today: **Mediation Policies** – but first, one leftover Security policy to close out.

JSON Threat Protection (recap)

Used to do operations on **JSON Incoming Content** – this policy will be used specifically when the incoming payload is JSON (the JSON-content counterpart to XML Threat Protection from Day 57).

Regular Expression Protection

Suppose incoming data – in the **header, query params, payload,** or **body** – needs to be checked against an **expression.** If it doesn't match, that's treated as an **expression error** (*"if we get Regular Expression..."* – i.e. if the configured pattern catches something unexpected/malicious in the content).

Demo scenario: a header contains a **threat (string)** – e.g. a specific flagged **word** – and the policy **throws an error** when it's found.

Regular Expression → **Validate** → **particular content** – for this, we use the **Regular Expression Protection** policy.

Similar to XML or JSON Threat Protection – we can also use Regular Expression Threat Protection, which helps you validate the **payload and also other data** (headers, query params, etc.) against a pattern.

Demo done: Regular Expression Threat Policy – configured on a **header**, with an **action** (**Keep** or similar) and a **threat (value)** to check for – resulting in an **Error** when matched.

With this, Security Policies are fully closed out.

2. Now, Mediation Policies – the full list

- **1.** Access Entity
- **2.** Assign Message
- **3.** Extract Variables
- **4.** JSON to XML
- **5.** Key Value Map Operations
- **6.** Raise Fault
- **7.** XSL Transform

(Item 7 corrected above from "XML Transformation" to the official name, "XSL Transform.")

3. Assign Message Policy

It's like a Content Modifier (the CPI equivalent) – it lets you **set headers** and **payload.**

Different operations available within Assign Message:

- **Set Payload**
- **Add Payload**
- **Copy**, **Remove**, **Add**, **Set** – the four core operation types, applicable to headers, query params, and payload alike.

4. Access Entity Policy

Access Entity, in API Management, works against a set of **standard entities.**

Question: what is a **Product name?**

Standard entities Access Entity can read:

- **Application name**
- **API Product name**
- **Consumer Key**
- **Developer name**
- **Company name**
- **Company developer name**

To read any of these, you use the **Access Entity** policy.

Scenarios covered in class:

- **Read the Developer name from the current API Key.** (Important: the **API key needs to be passed** for this to work.)
- **Access Product details.**

Demo: pass the **apikey** in the **header** (via Postman).

5. Extract Variables Policy

Extract Variables → extracts a **particular variable** out of the request/response content.

Demo: extracting the **Developer ID.**

6. Putting several Mediation + Security policies together (PreFlow)

Example PreFlow sequence drawn in class, chaining multiple policies on a single API Proxy:

```
PreFlow: Verify Auth -> Regular Expression -> Access Entity -> Extract (Variables)
-> Assign Message
```

This shows how a real proxy typically **layers several policies together** in sequence: first authenticate the caller, then validate content against a pattern, then pull entity/profile data, extract specific values from it, and finally shape the outgoing message.

7. JSON ↔ XML conversion, and XSL Transform

- **XML to JSON** – converts an XML payload to JSON. Requires its own **configuration.**
- **JSON to XML Converter** – converts a JSON payload to XML (the reverse direction). Also requires **configuration.**
- **XSL Transform** – converts XML using an XSLT stylesheet, into another format (XML, HTML, or plain text). **Allowed for one or two fields** at a time – using it for large-scale, complex transformations is **not encouraged** (that's squarely CPI's job, via proper Message Mapping).

8. Raise Fault Policy

Raise Fault – used for a **particular condition or scenario** that is **not satisfied**, returning a **custom error** back to the caller instead of letting the proxy fail in a generic, unhelpful way.

Demo scenario: Basic Auth – Username & Password **not matched** → **Raise Fault** returns a **Custom Error** explaining exactly what went wrong.

Quick-recall summary

- Closed out Security Policies with **Regular Expression Protection** – same validation idea as XML/JSON Threat Protection, applied via a pattern match against headers/query params/payload.
- 7 Mediation Policies: **Access Entity, Assign Message, Extract Variables, JSON to XML, Key Value Map Operations, Raise Fault, XSL Transform.**
- **Assign Message** = APIM's Content Modifier equivalent; operations: **Copy, Remove, Add, Set.**
- **Access Entity** reads standard entities (App, API Product, Consumer Key, Developer, Company, Company Developer) – needs the **API key** passed in.
- **Extract Variables** pulls a specific value out of a request/response.
- **XSL Transform** handles small, 1–2 field XML conversions only – not a substitute for CPI's full Message Mapping.
- **Raise Fault** returns a custom, meaningful error when a specific condition isn't met (e.g. Basic Auth mismatch).